Principles of Programming Languages 2026, Semester B

Home Assignment 1

Responsible TA: Rotem Shaked

Publication Date: 26/03/2026

Submission Date: 16/04/2026

Preliminaries

Structure of a TypeScript Project

The template of every TypeScript assignment will contain two important files:

- `package.json` - contains the metadata and package dependencies of the project.
- `tsconfig.json` - specifies the TypeScript compiler options.

Before starting to work on any assignment, open a command prompt in your assignment folder and run
`npm install`

When you do so, the Node Package Manager (`npm`) will read the `package.json` file to determine which packages and dependencies are required for the project, and it will download and install them into a folder called `node_modules`.
In addition, it will create a file called `package-lock.json` which lists the exact version of all the packages that have been installed.

The `tsconfig.json` file controls the way the TypeScript compiler (`tsc`) analyzes and type-checks the code in this project.
For all the assignments we will use the strongest form of type-checking, which is called the "strict" mode of the `tsc` compiler.

Important:
Do not delete or change these files (e.g., install new packages or change compiler options), as we will run your code against our own copy of those files, exactly the way we provided them.
If you change these files, your code may run on your machine but not when we test it, which may

lead to a situation where you believe your code is correct, but you would fail to pass compilation when we grade the assignment (which means a grade of zero).

Testing Your Code

Every TypeScript assignment will have `Jest` as a global dependency for testing purposes. In order to run the tests, save your tests in the `test` directory in a file ending with `.test.ts` and run `npm test` from a command prompt. This will activate the execution of the tests you have specified in the test file and report the results of the tests in a nice format.

An example test file `assignmentX.test.ts` might look like this:

```
import { sum } from "../src/assignmentX";

describe("Assignment X", () => {
  it("sums two numbers", () => {
    expect(sum(1, 2)).toEqual(3);
  });
});
```

Every function you want to test must be `export`-ed, so that it can be `import`-ed in the `.test.ts` file (and by our automatic test script when we grade the assignment).
For example, in `assignmentX.ts`:

```
export const sum = (a: number, b: number) => a + b;
```

You are given some basic tests in the `test` directory, just to make sure you are on the right track during the assignment.

What to Submit

You should submit a zip file called `<id1>_<id2>.zip` which has the following structure:

```
/
├── part1.md
├── src/
│   ├── part2.ts
│   ├── part3.ts
│   └── part4.ts
```

Please noice that part 1, which is the theoretical part, should be submitted in the Markdown format, which is a simple text-based markup language. See here for more details.

Make sure that when you extract the zip (using `unzip` on Linux), the result is flat, i.e., not inside a folder (the file `part1.md` is in the root directory).
This structure is crucial for us to be able to import your code to our tests.
Also, make sure the file is a `.zip` file, not a RAR or TAR or any other compression format.

Grading

This assignment will be evaluated through **frontal examination** (in-person review). You will be required to demonstrate a comprehensive understanding of your implementation and be prepared to explain your code, design decisions, and approach in detail.

Part 1: Theoretical Questions

Submit the solution to this part as `part1.md`.

[25 points] Question 1.1

1. Explain in simple words the following programming paradigms:
    i. [5 points] Imperative
    ii. [5 points] Object Oriented
    iii. [5 points] Functional
2. [5 points] How does the object oriented paradigm improve over the imperative paradigm?
3. [5 points] How does the functional paradigm improve over the object oriented paradigm?

[10 points] Question 1.2

Consider the following TypeScript function, which calculates the average price of all discounted products in a given inventory.

```
type Product = {
    name: string;
    price: number;
    discounted: boolean;
}

const getDiscountedProductAveragePrice = (inventory: Product[]): number => {
    let discountedPriceSum = 0;
    let discountedProductsCount = 0;

    for (const product of inventory) {
        if (product.discounted) {
            discountedPriceSum += product.price;
            discountedProductsCount++;
        }
    }

    if (discountedProductsCount === 0) {
        return 0;
    }

    return discountedPriceSum / discountedProductsCount;
}
```

This function uses an imperative approach with loops and conditional statements.

Refactor the function `getDiscountedProductAveragePrice` to adhere to the Functional Programming paradigm. Utilize the built-in array methods `map`, `filter`, and `reduce` to achieve the same functionality without explicit iteration and conditional checks.
Write the new function under the name `getDiscountedProductAveragePriceFP`.

Important: the new function should have the same signature.

Note: there are no tests for this question, and it will not be executed. The task here is to write the code in a functional way.

[18 points] Question 1.3

Write the most general type for each expression, using type variables where applicable.
Guidelines:

- Arrays must be homogeneous.
- Arithmetic operations must be performed on numbers.
- Use generics where possible.
- Avoid using `any`.

1. [3 points] `(x, y) => x.some(y)`
2. [3 points] `x => x.map(y => y * 2)`
3. [3 points] `(x, y) => x.filter(y)`
4. [3 points] `x => x.reduce((acc, cur) => acc + cur, 0)`
5. [3 points] `(x, y) => x ? y[0] : y[1]`
6. [3 points] `(f,g) => x => f(g(x+1))`

Part 2: TypeScript & Functional Programming

Instructions

In part 2 and part 3, replace every instance of the word `undefined` in the template with your code or typing.

Write the functions for the following questions in TypeScript in the file `src/part2/part2.ts`.
One of the assignment's dependencies is the `Ramda` library shown both in class and the practical session, and you may use it freely.
Make sure to write your code using type annotations, and adhering to the Functional Programming paradigm, specifically do not use `push`, `pop`, `shift`, `unshift`, `splice`, `sort`, `reverse`, `fill` on arrays, or any other mutating functions.

You may use helper functions as much as you want, but they must follow the same constraints as above.

You are also given a helper function `stringToArray` which takes a string and returns an array of the characters that make up the string. For example:

```
stringToArray("Hello!"); // ==> [ 'H', 'e', 'l', 'l', 'o', '!' ]
```

You are encouraged to use Ramda's `pipe` function, which takes a list of functions and returns a function which "pipes" the functions one after the other.
It is similar to `compose`, but the order of applications of functions is from left to right (as opposed to right to left in `compose`).
For example:

```
import { pipe } from "ramda";

const f = pipe(
    (x: number) => x * x,
    (x: number) => x + 1,
    (x: number) => 2 * x
);

f(5); // ==> 52
```

Remember that it is crucial you do *not* remove the `export` keyword from the code in the given template.

[5 points] Question 2.1

Write a function `countVowels` that takes a string as input and returns the number of vowels in the text. Reminder: vowels are one of 'a', 'e', 'i', 'o', 'u', either uppercase or lowercase. For example:

```
countVowels("This is SOME Text"); // ==> 5
```

[5 points] Question 2.2

Write a function `isPalindrome` that takes a string and returns a boolean value whether the string is a palindrome or not.
A palindrome is a word, phrase, number (a string containing digits), or other sequence of characters that reads the same forward and backward (ignoring spaces, punctuation, and capitalization).

Important:
You are not allowed to use the `reverse` function on arrays or strings.

```
console.log(isPalindrome("A man, a plan, a canal, Panama!")); // ==> true
console.log(isPalindrome("Not a palindrome")); // ==> false
```

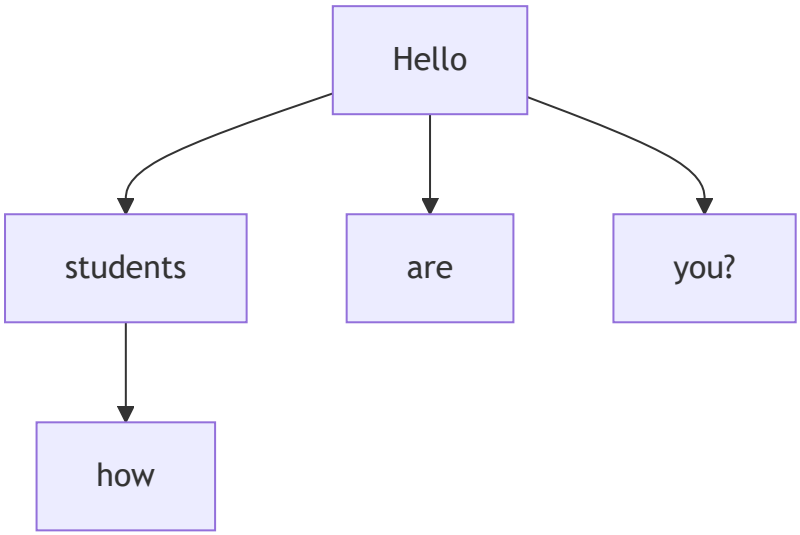
[5 points] Question 2.3

Given the following type:

```
type WordTree = {
    root: string;
    children: WordTree[];
}
```

Complete the function `treeToSentence`.
This function gets a `WordTree` as an argument and returns a string made of all the words in the tree, concatenated to each other, in pre-order traversal, separated by a single space.

Example:
Given this tree:




```
const t1: WordTree = {
    root: "Hello",
    children: [
        {
            root: "students",
            children: [
                {
                    root: "how",
                    children: []
                }
            ]
        },
        {
            root: "are",
            children: []
        },
        {
            root: "you?",
            children: []
        },
    ]
}

treeToSentence(t1); // ==> Hello students how are you?
```

[15 points] Part 3: A Gentle Introduction to Monads

What is a monad?
According to Wikipedia:

"In functional programming, a **monad** is a design pattern that allows structuring programs generically while automating away boilerplate code needed by the program logic.
Monads achieve this by providing their own data type (a particular type for each type of monad), which represents a specific form of computation, along with one procedure to wrap values of any basic type within the monad (yielding a **monadic value**) and another to compose functions that output monadic values (called **monadic functions**)."

During the semester, we will use two such monads: the `Result<T>` monad (used to deal with computations that may fail) and the `Optional<T>` monad (used when a computation might not yield a value).

The main function used with monads is the `bind` function (also called `chain` and `flatmap` in other languages, and by the `>>=` operator in Haskell).
`bind` is used to compose two monads in a way that makes sense in the context of the specific monad.

In your solution, in addition to `Result<T>` and `State<S, A>`, use only TypeScript constructs and types that are functional, i.e., under the same constraints as in Part 2, and that were covered in class.

When All Else Fails

We are going to get very familiar with the `Result<T>` monad in our interpreters' code, so to get up and running with using it and getting to know its constructors and its `bind` function, we will convert a function that throws an error to a function that uses `Result<T>`.

1. Read the code in `src/lib/result.ts`. Try to understand how `bind` composes two `Result<T>` values.
2. In `src/part3.ts`, you are given this code:

```typescript
/* Library code */
const findOrThrow = <T>(pred: (x: T) => boolean, a: T[]): T => {
    for (let i = 0; i < a.length; i++) {
        if (pred(a[i])) return a[i];
    }
    throw "No element found.";
}

/* Client code */
const returnSquaredIfFoundEven_v1 = (a: number[]): number => {
    try {
        const x = findOrThrow(x => x % 2 === 0, a);
        return x * x;
    } catch (e) {
        return -1;
    }
}
```

1. Write a *generic pure function* `findResult` which takes a predicate and an array, and returns an `Ok` on the first element that the predicate returns `true` for, or a `Failure` if no such element exists.
2. Only using `bind`, write a function `returnSquaredIfFoundEven_v2` that uses `findResult` to return an `Ok` on the first even value squared, or a `Failure` if no even numbers exist.
3. Only using `either` (see `src/lib/result.ts`), write a function `returnSquaredIfFoundEven_v3` that uses `findResult` to return the first even value squared, or a `-1` if no even numbers exist.

[20 points] Part 4: Programming in L3

Write the functions for the following questions in L3 in the file `src/part4.l3`.
Since L3 is a subset of Scheme, and to make things easier for you, you can use DrRacket or an online Scheme interpreter to test your code.
You may add auxiliary procedures to all questions.
Do not forget to write a contract for each of the above procedures.

```
; Signature:
; Type:
; Purpose:
; Pre-conditions:
; Tests:
```

[5 points] Question 4.1

Write an L3 procedure `last-item`, which gets a list and returns the last item in the list.
You may assume that the list is not empty.
For example:

```
(last-item '(1 2 3))  => 3
```

[5 points] Question 4.2

Write an L3 procedure `remove-last-item`, which gets a list and returns the list without the last item.
You may assume that the list is not empty.
For example:

```
(remove-last-item '(1 2 3)) => '(1 2)
```

[5 points] Question 4.3

Write an L3 procedure `rotate-nth`, which gets a list and a non-negative number `n`, and rotates the list `n` times to the right.
Examples:

```
(rotate-nth '(1 2 3) 1) => (3 1 2)
```

[5 points] Question 4.4

Write an L3 procedure `deep-reverse`, which gets a nested list and returns a list with all elements reversed at every level of nesting.
For example:

```
(deep-reverse '(1 2 3))           => (3 2 1)
(deep-reverse '(1 (2 3) (4 5)))   => ((5 4) (3 2) 1)
(deep-reverse '())                => ()
```

Good Luck and Have Fun!